

DEVELOPPEMENT D'APPLICATIONS WEB

HTML et CSS

Contenu de la partie HTML et CSS

I. HTML

1. Notion de balise
2. Types d'éléments HTML
3. Balises de base
4. Balises de structuration
5. Balises sémantiques
6. Balises multimédia
7. Les tableaux
8. Les formulaires

Contenu de la partie HTML et CSS

I. CSS

1. Syntaxe
2. Types d'intégration du CSS
3. Sélecteurs simples, complexes et pseudo-classes
4. Unités de mesure
5. Codification des couleurs
6. Marging et padding
7. Les propriétés de texte, alignement du texte, les fonts
8. Bordures et ombres, propriétés des images
9. Couleurs de fond, background
10. Types de positionnement
11. FlexBox
12. Les animations et les transitions
13. Le responsive design
14. Introduction aux frameworks CSS

HTML

Langage HTML: Définition

HTML signifie « HyperText Markup Language ». Il est utilisé afin de créer et de représenter le **contenu d'une page web et sa structure**. D'autres technologies sont utilisées avec HTML pour décrire la **présentation** d'une page (**CSS**) et/ou ses fonctionnalités **interactives** (**JavaScript**).



Langage HTML: Squelette d'une page HTML



Langage HTML: Notion de balise

- Une balise est un élément de texte encadrée par les caractères (<) et (>)
- Une balise correspond à un élément du code HTML d'une page Web

```
<balise attribut="valeur">Contenu</balise>
```

Balise ouvrante

On change ces valeurs pour modifier l'apparence de l'objet

Balise fermante

Langage HTML: Notion d'encapsulation

Les balises doivent être correctement encapsulées :

```
<p>
  <b>
    Paragraphe
  </b>
</p>
```

Exemple 1 : Code correct

```
<p>
  <b>
    Paragraphe
  </p>
</b>
```

Exemple 2 : Code incorrect

```
<div>
  <h1>titre</h1>
  <h2>un sous titre</h2>
  <p>un paragraphe avec
    <em>des mots importants</em>
    pour terminer avec une image
    
  </p>
</div>
```

Exemple 3 : Code correct

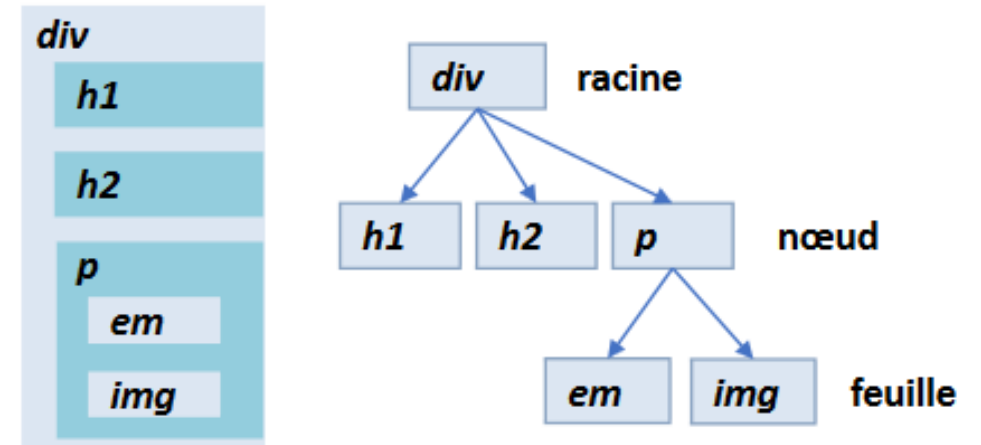


Figure 10 : Encapsulation des balises HTML

Langage HTML: types d'éléments HTML

- Il existe deux types d'éléments : **bloc** et **inline**
- Les éléments « bloc » s'étendent en largeur pour remplir totalement l'espace offert par son conteneur.
- Les éléments « inline » se suivent en ligne

Bloc

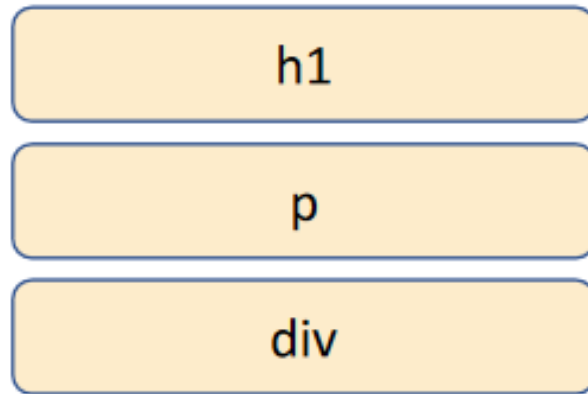


Figure 11 : Exemples d'éléments « bloc »

Inline

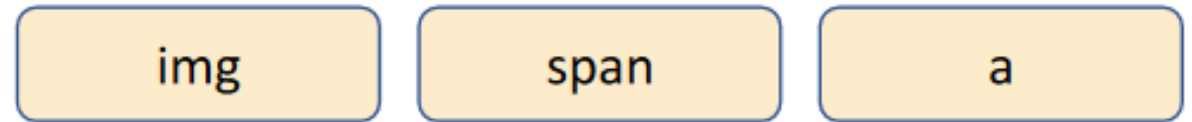


Figure 12 : Exemples d'éléments « inline »

Langage HTML: types d'éléments HTML

Éléments de niveau bloc en HTML :

<address>	<article>	<aside>	<blockquote>	<canvas>	<dd>	<div>
<dl>	<dt>	<fieldset>	<figcaption>	<figure>	<footer>	<form>
<h1> - <H6>	<header>	<hr>		<main>	<nav>	<noscript>
	<p>	<pre>	<section>	<table>	<tfoot>	
<video>						

Éléments de niveau inline en HTML :

<a>	<abbr>	<acronym>		<bdo>	<big>	
<button>	<cite>	<code>	<dfn>		<i>	
<input>	<kbd>	<label>	<map>	<object>	<output>	<q>
<samp>	<script>	<select>	<small>			<sub>
<sup>	<textarea>	<time>	<tt>	<var>		

Langage HTML: La balise <meta>

- Les métadonnées méta sont des données (informations) sur les données.
- La balise <meta> fournit des métadonnées sur le document HTML
- Les métadonnées ne seront pas affichées sur la page, mais seulement interprétables par le navigateur

Exemples de balises meta :

```
<meta charset="UTF-8">
```

Spécifie le format d'encodage du contenu.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Cet élément est indispensable pour avoir une page web responsive (qui s'adapte à la taille de l'écran utilisé).

```
<meta name="author " content=" x y z ">  
<meta name="description " content=" Cours web ">  
<meta name="keywords " content=" HTML,CSS,JavaScript ">
```

Ces éléments fournissent des informations sur le document html (utilisées par les moteurs de recherche)

Langage HTML: Balises de base

Les paragraphes en HTML:

- La balise <p> désigne un nouveau paragraphe
- L'attribut "title" permet d'afficher un infobulle quand la souris survole le paragraphe.

Exemple de déclaration d'un paragraphe :

```
<p title="infobulle sur le paragraphe">  
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor  
    incididunt ut labore et dolore magna  
    aliqua. Enim tortor at auctor urna. Dignissim enim sit amet venenatis. Varius sit  
    amet mattis vulputate enim. Urna et  
    pharetra pharetra massa massa.  
</p>
```

Langage HTML: Balises de base

Les titres en HTML:

Il existe 6 niveaux de titres (headers) en HTML. Les balises <h1>, <h2>, ...<h6> permettent l'affichage de ces titres

```
<h1>Niveau1</h1>  
<h2>Niveau2</h2>  
<h3>Niveau3</h3>  
<h4>Niveau4</h4>  
<h5>Niveau5</h5>  
<h6>Niveau6</h6>
```

Niveau1

Niveau2

Niveau3

Niveau4

Niveau5

Niveau6

Langage HTML: Balises de base

Les listes ordonnées et non ordonnées:

Exemple de liste non ordonnée

```
<ul>
  <li>Élément1</li>
  <li type="circle">Élément2</li>
  <li type="square">Élément3</li>
</ul>
```

- Élément1
- Élément2
- Élément3

Figure 16 : Liste non ordonnée

Exemple de liste ordonnée

```
<ol>
  <li type="I">Élément1</li>
  <li type="i">Élément2</li>
  <li type="A">Élément3</li>
  <li type="a">Élément4</li>
</ol>
```

- I. Élément1
- ii. Élément2
- C. Élément3
- d. Élément4

Figure 17 : Liste ordonnée 1

Exemple de liste ordonnée

```
<ol>
  <li>Élément1</li>
  <li>Élément2</li>
  <li value="1">Élément3</li>
  <li>Élément4</li>
</ol>
```

1. Élément1
2. Élément2
1. Élément3
2. Élément4

Langage HTML: Balises de base

Les liens:

La balise <a> permet de créer un lien hypertexte.

Les principaux attributs des liens hypertextes sont :

- href : la cible du lien
- title : affiche une info bulle sur le lien.
- Target :
 - _blank : ouverture de la page cible dans une nouvelle fenêtre.
 - _parent : ouverture de la page cible dans le cadre parent
 - _self : (valeur par défaut), ouverture de la page cible dans le cadre d'appel.
 - _top : ouverture de la page cible dans la fenêtre hôte (par-dessus le FRAMESET)

Langage HTML: Balises de structuration

Les balises <div>:

La balise HTML <div> (ou division) est un conteneur générique : elle peut contenir n'importe quel élément HTML.

Cet élément, de type bloc, n'a aucun effet sur le contenu ou la mise en page tant qu'il n'est pas mis en forme d'une manière explicite (en utilisant un style CSS).

<div> est très souvent utilisé avec la mise en page CSS d'une page Web.

Les balises :

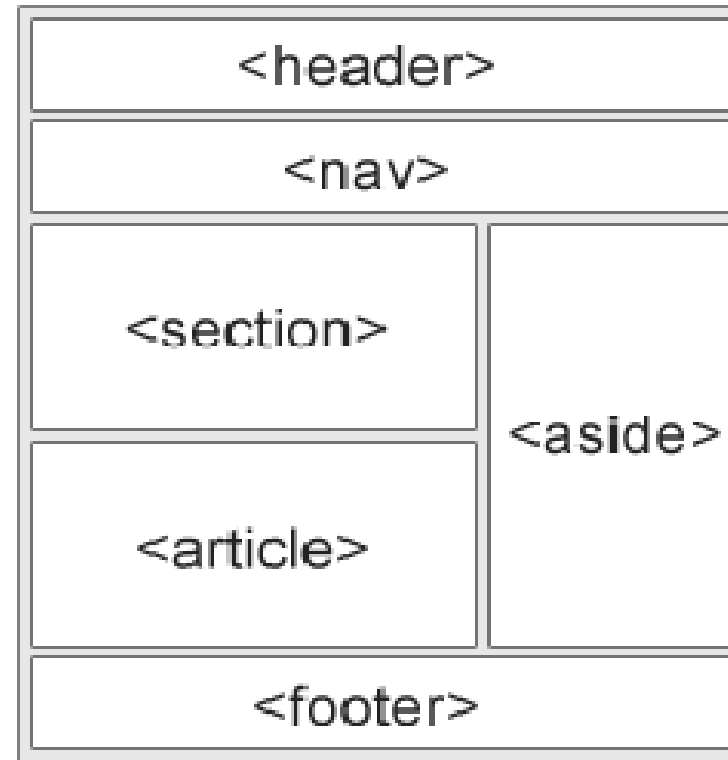
La balise ne produit aucun effet visuel mais sert à appliquer un effet de style CCS aux éléments qu'elle contient (un texte par exemple)

Langage HTML: Balises sémantiques

Plusieurs sites Web utilisent les balises comme : `<div id="nav">`, `<div class="header">`, ou `<div id="footer">`, pour indiquer les liens de navigation, en-tête, pied de page.

HTML5 propose des éléments sémantiques permettant de définir clairement les différentes parties d'une page web :

- `<header>`
- `<nav>`
- `<section>`
- `<article>`
- `<aside>`
- `<figure>` / `<figcaption>`
- `<footer>`
- `<details>` / `<summary>`
- `<mark>`
- `<time>`



Langage HTML: Balises multimédia (image, audio, vidéo)

Les images:

La balise permet d'insérer une image dans une page web.

- L'attribut src définit le chemin de l'image source
- l'attribut alt le texte alternatif si l'image n'est pas affichée.
- Les attributs width et height définissent la taille de l'image (en pixels)

Exemple :

```
  

```

Langage HTML: Balises multimédia (image, audio, vidéo)

La balise <audio>:

L'élément <audio> permet de lire un fichier audio dans la page HTML. Les attributs utilisés avec cette balise sont :

- Controls : pour afficher les boutons play, stop, preview...
- Autoplay : pour démarrer la lecture automatique de l'audio lors du chargement de la page web.

Le navigateur essaie de lire la première source. En cas d'erreur, il passe à la deuxième source. Si le navigateur ne prend pas en charge l'élément <audio>, il affiche le message en bas.

Exemple :

```
<audio controls autoplay>
  <source src="son.ogg" type="audio/ogg">
  <source src="son.mp3" type="audio/mpeg">Votre navigateur ne supporte pas cette balise
</audio>
```

Langage HTML: Balises multimédia (image, audio, vidéo)

La balise <video>:

L'élément <video> permet d'incorporer une vidéo dans une page Web. Les attributs utilisés avec cette balise sont :

- Controls : pour afficher les boutons play, stop, ...
- Autoplay : pour démarrer la lecture automatique de la vidéo lors du chargement de la page web.
- Poster : spécifie une image à afficher pendant le téléchargement de la vidéo. Si cet attribut n'est pas renseigné, la première image de la vidéo sera utilisée à sa place.

Le navigateur essaie de lire la première source. En cas d'erreur, il passe à la deuxième source. Si le navigateur ne prend pas en charge l'élément <video>, il affiche le message en bas.

Exemple :

```
<video width="320" height="240" controls autoplay poster="/images/poster.gif">  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogg" type="video/ogg">Votre navigateur ne supporte pas la balise  
  "video".  
</video>
```

Langage HTML: Balises multimédia (image, audio, vidéo)

La balise <iframe>:

La balise <iframe> permet d'afficher une ressource distante sur le web (page HTML, vidéo, ...)

Exemple :

```
<iframe width="640" height="360"  
src="https://www.youtube.com/embed/Ma1vSev07ug"></iframe>
```

Langage HTML: Balises multimédia (image, audio, vidéo)

La balise <object>:

L'élément <object> est une solution polyvalente pour inclure des objets génériques.

Exemple 1 : intégrer un document HTML dans un autre document HTML

```
<object data = "data/test.html" type = "text/html" width = "300" height = "200">  
  alt : <a href = "data/test.html">test.html</a>  
</object>
```

Exemple 2 : intégrer un document PDF dans un document HTML

```
<object data = "data/test.pdf" type = "application/pdf" width = "300" height = "200">  
  alt : <a href = "data/test.pdf">test.html</a>  
</object>
```

Langage HTML: Les tableaux

La balise <table>:

Pour insérer un tableau, on utilise la balise **<table>**.

À l'intérieur de la balise <table>, la balise **<tr>** (table row) signifie une ligne.

Chaque ligne est composée d'une ou plusieurs cellules de données définies par la balise **<td>**

Pour l'entête du tableau, on utilise la balise **<th>** (table header).

L'attribut **border** permet de fixer la largeur des bordures des cellules.

Exemple :

```
<table border="1">
  <tr>
    <th>Nom</th>
    <th>Prénom</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>ENAANAI</td>
    <td>Adil</td>
    <td>40</td>
  </tr>
  <tr>
    <td>SAFIR</td>
    <td>Laila</td>
    <td>32</td>
  </tr>
  <tr>
    <td>HAMIM</td>
    <td>Mohamed</td>
    <td>39</td>
  </tr>
</table>
```

Langage HTML: Les tableaux

Espacement des cellules:

- L'attribut "**cellspacing**" permet de définir l'espacement entre les cellules du tableau.
- l'attribut "**cellpadding**" permet de définir l'espacement entre le contenu de la cellule et ses bords.

Exemple :

```
<table border="1" cellspacing=10  
cellpadding=5>
```

The diagram shows a table with three columns: 'Nom', 'Prénom', and 'Age'. The rows contain the following data: (ENAANAI, Adil, 40), (SAFIR, Laila, 32), and (HAMIM, Mohamed, 39). A red double-headed arrow between the 'Prénom' and 'Age' headers is labeled 'cellspacing' in red, indicating the space between the cells. A blue double-headed arrow between the text 'SAFIR' and the bottom border of its cell is labeled 'cellpadding' in blue, indicating the space within the cell.

Nom	Prénom	Age
ENAANAI	Adil	40
SAFIR	Laila	32
HAMIM	Mohamed	39

Langage HTML: Les tableaux

Fusion des cellules:

Les attributs **colspan** et **rowspan** indiquent sur combien de colonnes et de lignes s'étend la cellule.

Exemple 1: On souhaite que la première cellule du tableau prenne toute la largeur de la ligne. Elle doit donc déborder sur 3 cellules horizontales : colspan=3

```
<table border=1>
  <tr>
    <td colspan="3">cellule 1</td>
  </tr>
  <tr>
    <td>cellule 2</td>
    <td>cell 2</td>
    <td>cell 4</td>
  </tr>
</table>
```

cellule 1		
cellule 2	cel 3	cel 4

Langage HTML: Les tableaux

Fusion des cellules:

Exemple 2: On souhaite que la première colonne du tableau déborde sur 2 lignes :
rowspan=2

```
<table border="1">
  <tr>
    <td rowspan="2">cellule 1</td>
    <td>cell 2</td>
    <td>cel 3</td>
  </tr>
  <tr>
    <td>cell 4</td>
    <td>cell 5</td>
  </tr>
</table>
```

cellule 1	cel 2	cel 3
	cel 4	cel 5

Langage HTML: Les formulaires

Balise <form>:

Un formulaire est une section d'un document web qui permet à un utilisateur de fournir des informations via des contrôles interactifs

L'élément HTML< **form**> permet de déclarer un formulaire

L'attribut "action" spécifie la ressource qui va recevoir les données du formulaire pour les traiter (un script)

L'attribut "**method**" spécifie la méthode d'envoi des données

On distingue deux méthodes :

- **GET** : permet d'envoyer les données sous forme de chaîne de paramètres à travers l'URL. Les données sont visibles dans la barre d'adresse.
- **POST** : permet d'envoyer les données (invisibles) à travers une requête HTTP.

Exemple :

```
<form action="authentication.php" method="post">  
  <!-- Ici on insère les éléments du formulaire-->  
</form>
```

Langage HTML: Les formulaires

Champ de saisie <input>:

L'élément <input> permet de créer différents types d'entrée pour un formulaire, selon la valeur de l'attribut type.

Exemple :

```
<form action="authentification.php" method="post">  
  <input type="text" id="nom" name="nom">  
</form>
```

Types d'Input HTML:

- <input type="button">
- <input type="checkbox">
- <input type="color">
- <input type="date">
- <input type="datetime-local">
- <input type="email">
- <input type="file">
- <input type="hidden">

- <input type="image">
- <input type="month">
- <input type="number">
- <input type="password">
- <input type="radio">
- <input type="range">
- <input type="reset">
- <input type="search">

- <input type="submit">
- <input type="tel">
- <input type="text">
- <input type="time">
- <input type="url">
- <input type="week">

Langage HTML: Les formulaires

Type texte:

```
<form>
  <label for="pr">Prénom :</label><br>
  <input type="text" id="pr" name="pr"><br>
  <label for="nf">Nom de famille :</label><br>
  <input type="text" id="nf" name="nf">
</form>
```

Type password:

```
<form>
  <label for="nomutil">Nom d'utilisateur :</label><br>
  <input type="text" id="nomutil" name="nomutil"><br>
  <label for="pwd">Mot de passe :</label><br>
  <input type="password" id="pwd" name="pwd">
</form>
```

Langage HTML: Les formulaires

Champ de saisie <textarea>:

L'élément <textarea> permet d'afficher et saisir un texte en plusieurs ligne

Type radio:

```
<form>
  <input type="radio" id="mas" name="sexe" value="Masculin">
  <label for="mas">Masculin</label><br>
  <input type="radio" id="fem" name="sexe" value="Féminin">
  <label for="fem">Féminin</label><br>
</form>
```

Type number:

```
<form>
  <label for="quant">Quantité :</label>
  <input type="number" id="quant" name="quant" min="0" max="100" step="10" value="30">
</form>
```

Langage HTML: Les formulaires

Type checkbox:

```
<form>
  <input type="checkbox" id="vehicule1" name="vehicule1" value="velo">
  <label for="vehicule1"> J'ai un vélo</label><br>
  <input type="checkbox" id="vehicule2" name="vehicule2" value="voiture">
  <label for="vehicule2">J'ai une voiture</label><br>
  <input type="checkbox" id="vehicule3" name="vehicule3" value="bateau">
  <label for="vehicule3"> J'ai un bateau</label>
</form>
```

Type select:

```
<label for="langage">Sélectionner un langage de programmation:</label>
<select name="lg" id="langage">
  <option value="">--Choisir --</option>
  <option value="Python">Python</option>
  <option value="CSharp">C#</option>
  <option value="java">java</option>
</select>
```

Langage HTML: Les formulaires

Type button:

```
<input type="button" onclick="alert('Bonjour!')" value="Clic ici!">
```

Type range:

```
<form>  
  <label for="vol">Volume (entre 0 and 50) :</label>  
  <input type="range" id="vol" name="vol" min="0" max="50">  
</form>
```

Type date:

```
<form>  
  <label for="anniv">Anniversaire :</label>  
  <input type="date" id="anniv" name="anniv">  
</form>
```

CSS

Les feuilles de style en cascade, généralement appelées CSS de l'anglais « Cascading Style Sheets », forment un langage informatique qui décrit la **présentation** des documents HTML et XML.



CSS: Syntaxe

Le sélecteur est un élément clé qui indique sur quels éléments HTML vont s'appliquer les propriétés CSS



```
sélecteur {  
    propriété: valeur;  
    propriété: valeur;  
    ...  
}
```

C'est l'ensemble de paramètres de mise en forme à appliquer sur l'élément identifié par le sélecteur.

Exemple:

```
p {  
    color: red;  
}
```

CSS: Types d'intégration du CSS

Dans la même page HTML:

Pour intégrer une feuilles de style CSS dans une page HTML, il faut placer le code CSS dans la balise <style>.

Exemple :

```
<!DOCTYPE html>
<head>
<style>
  #myDiv1
  {
    width : 100px;
    height : 100px;
    border : 1px solid red;
    background-color : yellow;
  }
</style>
</head>
<body>
  <div id="myDiv1"></div>
</body>
</html>
```

CSS: Types d'intégration du CSS

Dans un fichier externe:

La balise <link > permet d'intégrer une feuilles de style CSS dans une page HTML

Exemple :

```
<!DOCTYPE html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div id="myDiv1"></div>
</body>
</html>
```



```
#myDiv1
{
  width : 100px;
  height : 100px;
  border : 1px solid red;
  background-color : yellow;
}
```

CSS: Sélecteurs simples

Les sélecteurs de type:

Ce sélecteur permet de cibler les éléments qui correspondent au nom indiqué.
(nom de la balise)

```
div
{
    background-color : yellow;
}
```

Les sélecteurs d'identifiant "#":

Le sélecteur de type identifiant permet de cibler un élément d'un document en fonction de la valeur de son attribut id

```
<table border="1">
  <tr><th>Nom</th><th>Prénom</th><th>Age</th></tr>
  <tr><td>MANSOURI</td><td>Hassan</td><td>23</td></tr>
  <tr><td>SAFIR</td><td>Laila</td><td>22</td></tr>
  <tr id="sel"><td>BICHRI</td><td>Karim</td><td>21</td></tr>
  <tr><td>HAMIM</td><td>Mohamed</td><td>24</td></tr>
</table>
```

```
#sel
{
    background-color : rgba(255, 255, 0, 0.2);
}
```

Les sélecteurs de classe .:

Ce sélecteur de type « class » permet de cibler les éléments en fonction de la valeur de leur attribut class

```
<table border="1">
  <tr><th>Nom</th><th>Prénom</th><th>Age</th></tr>
  <tr class="impaire"><td>MANSOURI</td><td>Hassan</td><td>23</td></tr>
  <tr><td>SAFIR</td><td>Laila</td><td>22</td></tr>
  <tr class="impaire"><td>BICHRI</td><td>Karim</td><td>21</td></tr>
</table>
```

```
.impaire
{
    background-color : rgba(255, 255, 0, 0.2);
}
```

CSS: Sélecteurs complexes

Le sélecteur universel "*":

Ce sélecteur permet de cibler tous les nœuds d'un document

```
*  
{  
    color : brown;  
}
```

Les sélecteurs d'attribut []:

[attr] : cible un élément qui possède un attribut attr.

- **[attr=valeur]** : cible un élément qui possède un attribut attr dont la valeur est exactement valeur.
- **[attr^=valeur]** : cible un élément qui possède un attribut attr dont la valeur commence par valeur.
- **[attr\$=valeur]** : cible un élément qui possède un attribut attr dont la valeur se termine par valeur.
- **[attr*=valeur]** : cible un élément qui possède un attribut attr et dont la valeur contient au moins une occurrence de valeur dans la chaîne de caractères.

CSS: Sélecteurs complexes

Combinateur de voisin direct +

Exemple: sélectionne la balise p qui vient directement après la balise ul

```
ul + p
{
    color : red;
}
```

Sélecteurs enfant >

Exemple: sélectionne la balise span fille directe de la balise div

```
span
{
    background-color : white;
}
div > span
{
    background-color : blue;
}
```

Sélecteurs descendant " " (espace):

Exemple: sélectionne la balise li descendante de la balise li

```
li
{
    list-style-type : disc;
}
li li
{
    list-style-type : circle;
}
```

CSS: Pseudo-classes

input:checked: sélectionne tout élément `<input>` coché du formulaire

input:disabled: sélectionne chaque élément `<input>` désactivé

p:empty: sélectionne chaque élément `<p>` qui n'a pas d'enfant

input:enabled: sélectionne chaque élément `<input>` activé

input:invalid: sélectionne tous les éléments `<input>` avec une valeur invalide

p:last-child: sélectionne tous les éléments `<p>` qui sont les derniers enfants de leurs parents

:not(p): sélectionne tous les éléments qui ne sont pas des paragraphes

p:nth-child (2): sélectionne chaque élément `<p>` qui est le deuxième enfant de son parent

input:read-only: sélectionne les éléments <input> avec un attribut "readonly" spécifié

input:required: sélectionne les éléments <input> avec un attribut "required" spécifié

a:visited: sélectionnez tous les liens visités

a:active: sélectionne le lien actif

a:hover: sélectionne les liens sur lesquels la souris passe

input:focus: sélectionne l'élément <input> qui a le focus

p:first-letter: sélectionne la première lettre de chaque élément p

p:first-line: sélectionne la première ligne de chaque élément p

p:before: insérer du contenu avant chaque élément p

p:after: insérer du contenu après chaque élément p

CSS: Unités de mesure

Les unités absolues:

- Les pixels (px)
- Les pouces (in)
- Les centimètres (cm)
- Les millimètres (mm)

Les unités relatives au texte:

- **em** : proportionnelle à la taille de la police de l'élément parent ou du document. Par défaut, 1 em = 16 px si aucune taille de police n'est définie.
- **rem** : fait toujours référence à la taille de la police de l'élément racine. Elle dépend du font-size défini par défaut

Les unités relatives au viewport:

Les unités relatives au viewport permettent aux éléments de s'adapter à la taille de la fenêtre du navigateur.

- La hauteur du viewport (vh)
- La largeur du viewport (vw)

CSS: Codification des couleurs

Méthode	Exemple	Explication
Couleurs nommées	background-color : grey	La couleur est désignée par son nom en anglais. Il n'y a que 16 noms de couleurs normalisés par le W3C
Couleurs en hexadécimal	background-color : #808080	La couleur est désignée par son code hexadécimal : les deux premiers digits correspondent à la valeur de rouge, les deux suivants le vert et les deux derniers le bleu
Couleurs décomposées	background-color : RGB(128,128,128)	La couleur est définie par trois nombres décimaux qui peuvent prendre les valeurs de 0 à 255 indiquant respectivement le taux de rouge, le taux de vert et le taux de bleu

CSS: Marging et padding

Les propriétés **margin** et **padding** permettent de spécifier la marge **externe** et **interne** respectivement.

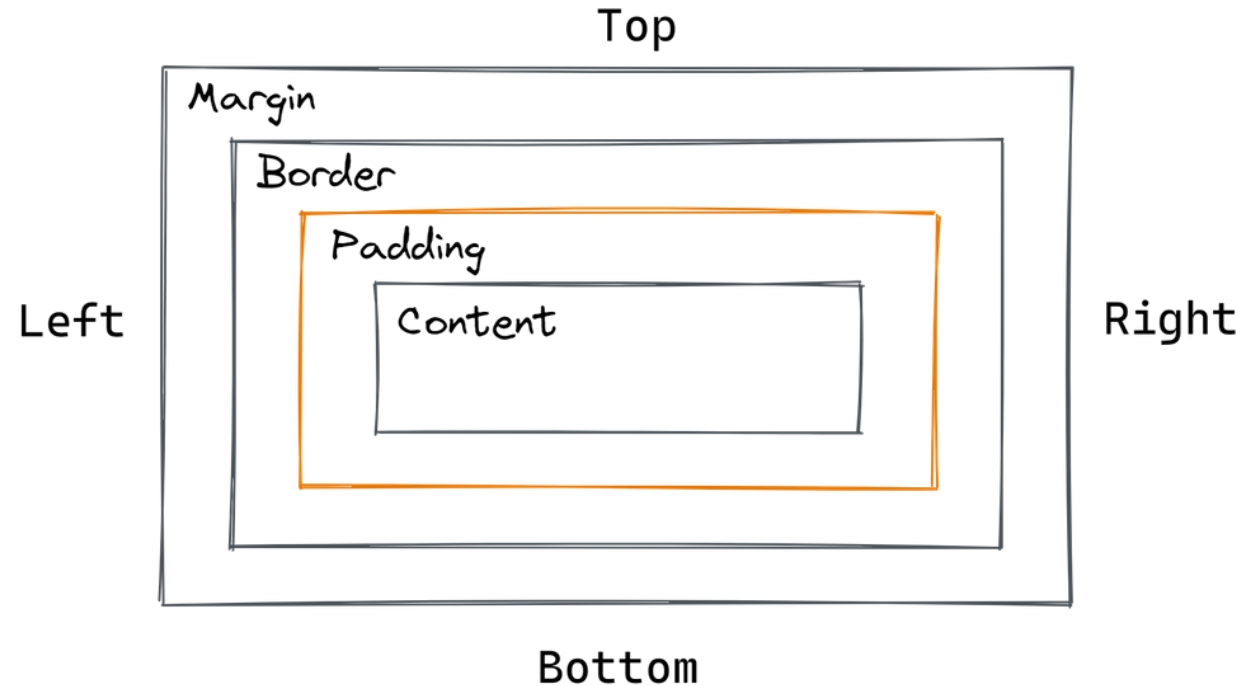
Les deux propriétés peuvent admettre soit:

- Une valeur: la même pour les quatre cotés
- Deux valeurs: la première pour (top et bottom), la deuxième (left et right)
- Trois valeurs: la première pour (top), deuxième (bottom), 3ème pour (left et right)
- Quatre valeurs: pour top, right, bottom, et left

Les propriétés margin-top, margin-left, padding-left,...etc peuvent aussi être utilisées pour spécifier la marge pour l'un des quatre cotés

Exemple:

```
div
{
  padding: 20px;
  margin: 20px;
}
```



CSS: Les propriétés de texte

- **font-style** : spécifie un texte en italique. Valeurs possibles : normal, italic, oblique.
- **font-weight** : permet de mettre en gras un texte. Valeurs possibles : de 100 à 900, normal, bold, bolder, lighter.
- **word-spacing** : l'espace entre les mots (en pixels).
- **letter-spacing** : l'espace entre deux lettres au sein d'un même mot (en pixels).
- **text-decoration** : spécifie la décoration du texte. Valeurs possibles : none, underline, overline , line-through, blink.
- **text-transform** : permet de contrôler la capitalisation du texte. Valeurs possible : none, uppercase (majuscules), lowercase (miniscule), capitalize (la première lettre de chaque mot sera en majuscule).
- **text-align** : permet d'aligner horizontalement le texte. Valeurs possibles: left, center, right, justify
- **text-indent** : taille du retrait de première ligne (en pixels)
- **line-height** : hauteur de ligne (en pixels).

CSS: Bordures et ombres

```
<div id="myDiv1"></div>
```

```
div
{
  width: 100px;
  height: 100px;
  background-color: yellow;
  border: 1px dashed red;
  border-radius: 10px;
  box-shadow: 10px 10px 5px gray;
}
```

Résultat :



Distance horizontale

Distance verticale

Rayon de l'arrondi

CSS: Propriétés des images

Propriétés **width** et **height** : définissent la taille de l'image

- Propriété **max-width** : permet à l'image de s'adapter à la taille de l'écran
- Propriété **opacity** : gère l'opacité de l'image
- Propriété **filter** : ajoute des effets visuels (comme le flou et la saturation) à l'image

CSS: Couleurs de fond

```
<p>Ceci est un texte colorié en bleu</p>
```

```
p {  
  color: blue;  
  font-size: 24px;  
  font-family: cursive;  
  background-color: yellow;  
}
```

Ceci est un texte colorié en bleu

L'élément image peut être inséré comme arrière plan d'un élément.

Les propriétés associées sont :

- **background-image** : url de l'image de fond
- **background-repeat** : mode de réplication de l'image de fond (repeat, repeat-x, repeat-y, no-repeat)
- **background-position** : position de l'image d'arrière plan (top, bottom, left, center, right)
- **background-size** (100 % 100 %)

CSS: Alignement du texte

```
h1
{
    text-align : center;
}
h2
{
    text-align : left;
}
h3
{
    text-align : right;
}
h4
{
    text-align : justify;
}
```

CSS: Les fonts

Taille du texte : `font-size`

Peut être spécifiée de 4 manières :

- En pixels : ou en mm, cm.
- En valeur relative : xx-small, x-small, small, medium, large, x-large xx-large.
- En em : par exemple, 1em signifie "Taille normale", 1.3em signifie un texte de taille 1,3 fois plus grande, 0.8em signifie que votre texte aura une taille 0,8 fois plus petite.
- En % : par exemple, 100% : le texte aura une taille "normale", 130% : le texte aura une taille correspondant à 130% de la taille normale

Type de la police: `font-family`

C'est une suite de noms de polices.

Cinq familles sont distinguées : serif (Times, etc.), sans-serif (Arial, Helvetica, etc.), cursive, monospace, Fantasy.

Les polices sont associées de préférence « en pile ». Si un visiteur du site n'a pas la première police sur sa machine, on passe automatiquement à la suivante et ainsi de suite.

Règle `@font-face` : permet d'importer un font personnalisé (extension .ttf)

CSS: Types de positionnement

Il existe trois mécanismes de positionnement de base en CSS :

1. Le flux normal : consiste à garder le flux normal des éléments selon leur ordre et leur type (bloc , inline)

2. La propriété « position » :

- Relative
- Absolue
- Fixe

3. La propriété « float » :

- Définit la position flottante d'un élément à droite (valeur right) ou à gauche (valeur left) dans un document (ou dans la boîte conteneur).
- Nettoyer les flottants : propriété clear

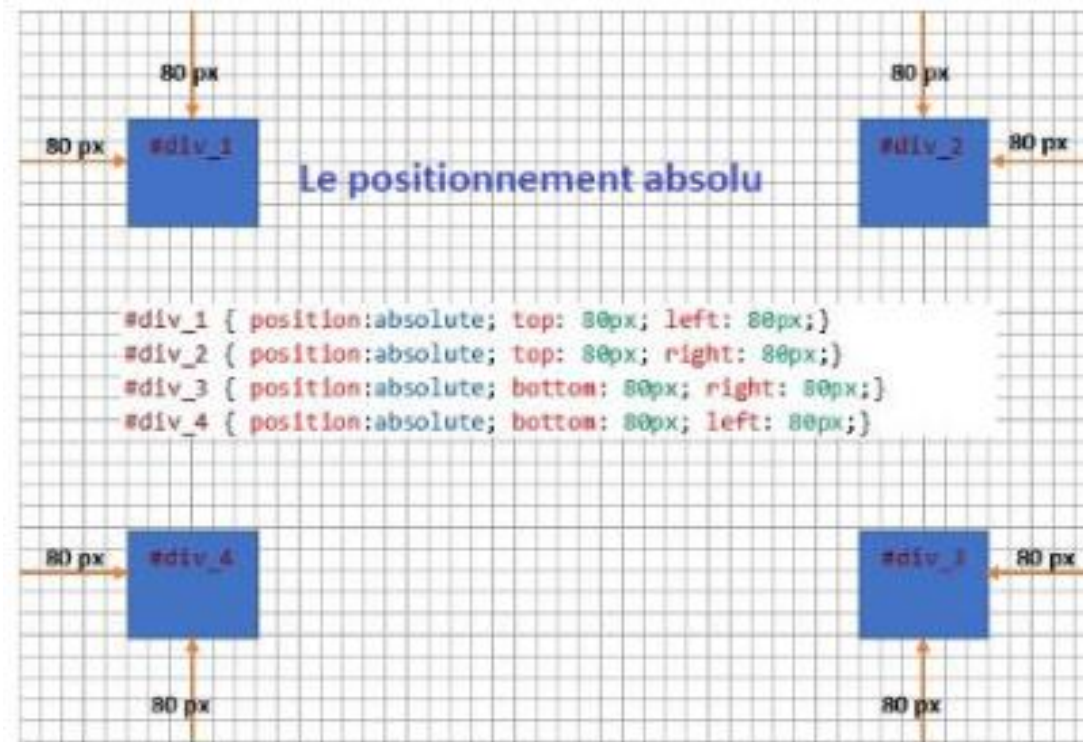
CSS: Types de positionnement

Le positionnement absolu:

Ce type de positionnement permet de placer un élément de la page exactement à l'emplacement souhaité.

- Une balise HTML avec une position absolue ne laisse aucun espace vide après qu'elle est positionnée
- On place une balise HTML en position absolue en fixant la valeur de la propriété position à "absolute"
- Les propriétés right, left, top et bottom sont alors utilisées pour définir l'emplacement de la balise HTML

Exemple :



CSS: Types de positionnement

Le positionnement relatif:

On place une div en position relative avec la valeur de la propriété : relative

La position relative d'un élément (div par exemple) est calculée d'après sa position originale dans la page (sa position dans le flux normal)

l'élément peut être déplacé vers la droite, la gauche, le haut ou le bas (top, left, bottom et right)

Le positionnement fixe:

Pareil que le positionnement absolu, mais le block reste toujours visible, même si on défile la page

La propriété CSS float:

La propriété float est utilisée pour positionner et formater le contenu.

Par exemple, faire flotter une image à gauche du texte dans un conteneur.

- La propriété float peut avoir les valeurs suivantes :
- left : L'élément flotte à gauche de son conteneur.
- right : L'élément flotte à droite de son conteneur.
- none : La valeur par défaut, l'élément ne flotte pas.
- inherit : L'élément hérite de la valeur flottante de son parent

Pourquoi utiliser Flexbox ?

La méthode Flexbox facilite la structuration et la mise en page souple et réactive, sans utiliser les marges et le positionnement.

La page est décomposée en un agencement de boîtes qui peuvent se suivre (boîtes voisines) et s'imbriquer (boîtes ancêtres).

En définissant les tailles et les interactions entre les différentes boîtes, on obtient une adaptation automatique des éléments de la page lorsqu'elle est redimensionnée.

Comment utiliser Flexbox ?

Le conteneur flex (flex container) ou bien la boîte dite « parent », contient les éléments flex (flex items) ou bien les boîtes « enfants ».

L'idée est définir, en utilisant les différentes propriétés CSS, l'ordre des "boîtes enfants" dans la "boîte parent".

CSS: FlexBox

Définir le conteneur flex (flex container):

Le conteneur est une balise html qui contient d'autres balises html.

Dans l'exemple ci-dessous, La conteneur correspond à la balise <div> pour laquelle on a défini la classe « .conteneur ».

Cette balise <div> contient deux balises <div> et une balise <figure>. Ces dernières deviennent alors des "éléments flex" (flex items) mis en évidence ici par la classe ".element".

```
<div class="conteneur">
  <div class="element">Élément 1</div>
  <figure class="element">Élément 2</figure>
  <div class="element">Élément 3</div>
</div>
```

La boîte se transforme en "conteneur flex" dès lors qu'on lui attribue la propriété CSS "flex" comme montré dans le code CSS suivant

```
.conteneur
{
  display : flex;
}
```

CSS: FlexBox

Définir les axes du conteneur flex avec la propriété flex-direction:

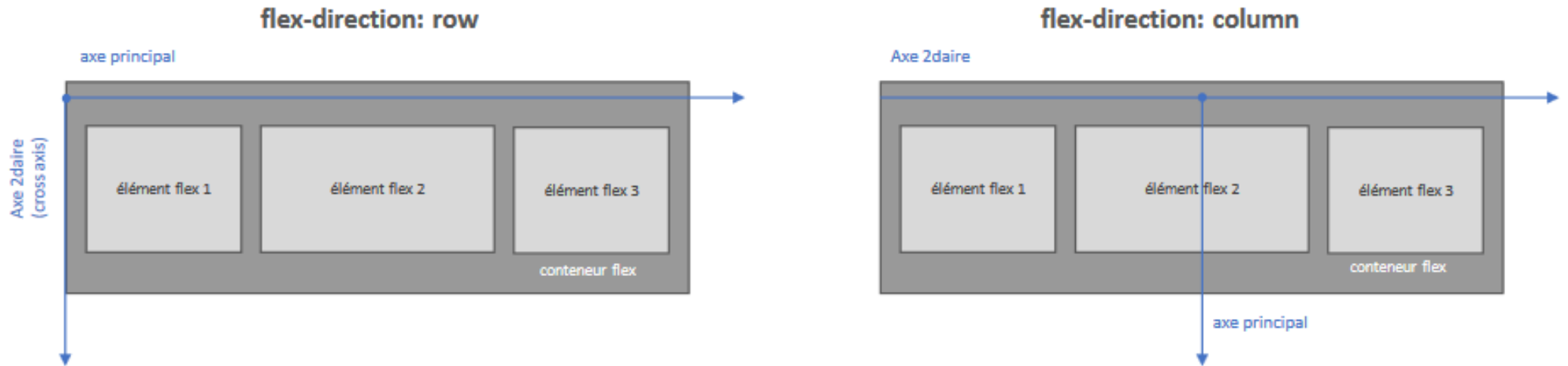
Axe principal : main axis

Le conteneur flex possède un axe principal (main axis) sur lequel les éléments flex peuvent se suivre. L'axe principal est défini par la propriété **flex-direction** qui peut prendre les valeurs suivantes :

- **row** : les éléments flex se suivent sur une ligne
- **row-reverse** : les éléments flex se suivent sur une ligne en ordre inversé.
- **column** : les éléments flex se suivent sur une colonne.
- **column-reverse** : les éléments flex se suivent sur une colonne en ordre inversé

Axe secondaire : cross axis

Le choix de l'axe principal impose la direction de l'axe secondaire (cross axis) qui lui est toujours perpendiculaire



CSS: FlexBox

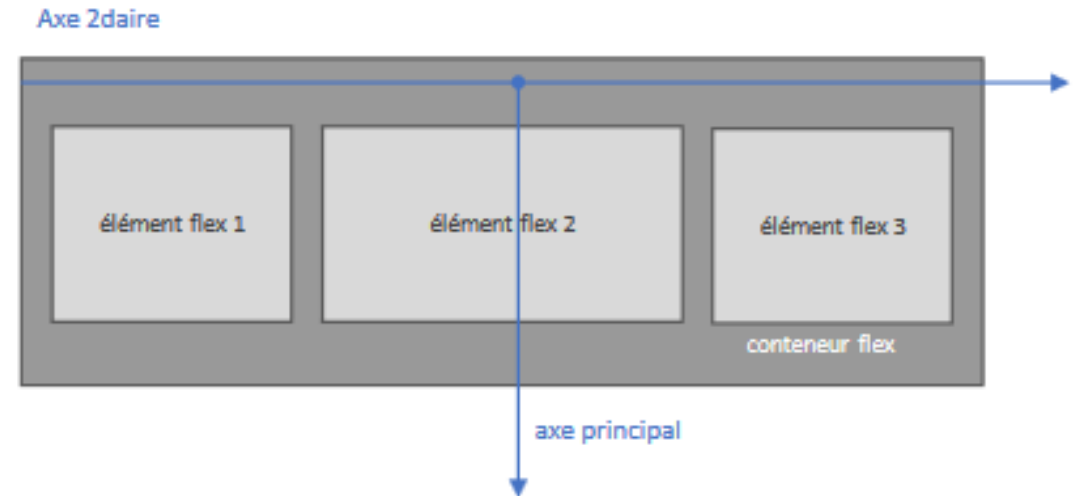
Le choix des axes est essentiel car leur orientation va déterminer le résultat de l'application des propriétés suivantes :

- **justify-content** : définit comment les éléments flex sont positionnés le long de l'axe principal
- **align-items** : définit comment les éléments flex sont positionnés le long de l'axe secondaire

flex-direction: row



flex-direction: column



CSS: FlexBox

Les différentes valeurs possibles pour les propriétés flex:

```
<div class="conteneur">
  <div class="element1">élément 1</div>
  <div class="element2">élément 2</div>
  <div class="element3">élément 3</div>
  <div class="element4">élément 4</div>
</div>
```

```
.conteneur
{
  background-color : blue;
  height : 230px;
  width : 500px;
  display : flex;
  flex-direction : row | column;
  align-items : stretch | flex-start | flex-end | center | baseline;
  justify-content : flex-start | flex-end | center | space-around | space-between | space-evenly;
}
```

Maitriser les passages à la ligne : la propriété **flex-wrap**:

La propriété "flex-wrap" peut prendre les valeurs suivantes :

- **flex-wrap : nowrap** - c'est la valeur initiale. Si les éléments flexibles sont trop larges pour tenir dans leur conteneur, ils débordent du conteneur
- **flex-wrap : wrap** - si les éléments flexibles sont trop larges pour tenir dans le conteneur, ils passent automatiquement sur la ligne suivante (du haut vers le bas)
- **flex-wrap : wrap-reverse** - si les éléments flexibles sont trop larges pour le conteneur, ils passent automatiquement sur la ligne suivante (du bas vers le haut)

La propriété raccourcie **flex-flow** combine flex-direction et flex-wrap. Elle peut, par exemple, prendre les valeurs "row wrap" ou "column wrap"

Les propriétés des éléments flex:

La propriété **order** permet de contrôler l'ordre dans lequel les éléments apparaissent dans le conteneur flex. Les valeurs possibles de cette propriété peuvent être soit des valeurs globales ("inherit", "initial", "unset " ou bien des valeurs entières positives ou négatives

- La propriété **flex-grow** permet de définir l'agrandissement possible d'un élément proportionnellement aux autres. La valeur de cette propriété est un nombre. Par exemple, si "flex-grow : 2" pour un élément, il occupe (si possible) deux fois plus d'espace que les autres
- La propriété **flex-shrink** définit la possibilité pour un élément de rétrécir. Cette propriété accepte uniquement des valeurs positives
- La propriété **flex-basis** définit la taille par défaut d'un élément. Elle peut prendre des valeurs définies (3 em, 20 px, auto), des valeurs globales (inherit, initial, unset), des valeurs définies par mots-clés (fill, max-content, min-content, fit-content) ou la valeur "content" qui calcule la taille automatiquement en fonction du contenu de l'élément
- La propriété **flex** est une propriété raccourcie combinant les propriétés "flex-grow", "flex-shrink" et "flex-basis"
- La propriété **align-self** correspond à l'application de la propriété "align-items" pour un seul élément

CSS: Les animations

Les transformations 2D:

Les transformations CSS 2D permettent de déplacer, faire pivoter, mettre à l'échelle et incliner des éléments

```
div {  
  width: 300px;  
  height: 100px;  
  background-color: cyan;  
  border: 1px solid black;  
}  
  
div:hover {  
  transform: translate(50px,100px);  
}
```

```
div {  
  width: 300px;  
  height: 100px;  
  background-color: cyan;  
  border: 1px solid black;  
}  
div:hover {  
  transform: rotate(20deg);  
}
```

CSS: Les animations

```
div {  
  width: 300px;  
  height: 100px;  
  background-color: cyan;  
  border: 1px solid black;  
}  
  
div:hover {  
  transform: translate(50px,100px);  
}
```

```
div {  
  width: 300px;  
  height: 100px;  
  background-color: cyan;  
  border: 1px solid black;  
}  
div:hover {  
  transform: scale(2,3);  
}
```

CSS: Les transitions

Les transitions CSS permettent de modifier les valeurs des propriétés sur une durée donnée.

Exemple : un élément <div> rouge de 100px * 100px subit un effet de transition pour la propriété width, d'une durée de 2 secondes

```
div
{
  width: 100px;
  height: 100px;
  background: red;
  transition: width 2s;
}
```

```
div:hover
{
  width: 300px;
}
```

Exemple avec plusieurs propriétés

```
div
{
  transition: width 2s, height 4s;
}
```

CSS: Propriétés d'animation

La propriété raccourcie (ou bien les propriétés détaillées correspondantes) est utilisée pour créer une animation CSS. Elle permet de configurer la durée, le minutage et d'autres détails de l'animation.

L'apparence visuelle de l'animation est définie en utilisant des règles CSS de mise en forme au sein de la règle `@keyframes`.

Les propriétés détaillées rattachées à la propriété raccourcie animation sont :

- **animation-delay** : Cette propriété définit le délai entre le chargement de l'élément et démarrage de l'animation.
- **animation-direction** : Cette propriété précise si l'animation doit alterner entre deux directions de progression (faire des allers-retours) ou recommencer au début à chaque cycle de répétition.
- **animation-duration** : Cette propriété définit la durée d'un cycle de l'animation.
- **animation-fill-mode** : Cette propriété indique les valeurs à appliquer aux propriétés avant et après l'exécution de l'animation.
- **animation-iteration-count** : Cette propriété détermine le nombre de répétition de l'animation. Pour répéter une animation infiniment, On utilise la valeur infinie.
- **animation-name** : Cette propriété déclare un nom à l'animation (utilisé comme référence à l'animation pour la règle `@keyframes`).
- **animation-play-state** : Cette propriété permet d'interrompre (« pause ») ou de reprendre l'exécution d'une animation.
- **animation-timing-function** : Cette propriété configure la fonction de minutage d'une animation.

CSS: Les keyFrames

La spécification d'un style CSS à l'intérieur de la règle @keyframes permet à une animation de passer progressivement du style actuel au nouveau style

```
@keyframes exemple
{
    from {background-color: red;}
    to {background-color: yellow;}
}
/* L'élément sur lequel s'appliquer l'animation */
div
{
    width: 100px;
    height: 100px;
    background-color: red;
    animation-name: exemple;
    animation-duration: 4s;
}
```

```
@keyframes exemple
{
    0% {background-color: red;}
    25% {background-color: yellow;}
    50% {background-color: blue;}
    100% {background-color: green;}
}
/* L'élément sur lequel s'appliquer l'animation */
div
{
    width: 100px;
    height: 100px;
    background-color: red;
    animation-name: exemple;
    animation-duration: 4s;
}
```

CSS: Adapter une page web au dispositif d'affichage

Introduction au Responsive Design:

Les pages Web peuvent être consultées à l'aide de nombreux types d'appareils : ordinateurs de bureau, tablettes et téléphones. Ainsi, les pages Web ne doivent pas omettre d'informations pour s'adapter aux appareils plus petits, mais plutôt adapter leur contenu pour s'afficher correctement sur n'importe quel appareil utilisé

- La conception Web réactive (Responsive Design) rend la page Web compatible et adaptable à tous les appareils. Elle est basée uniquement sur HTML et CSS (elle n'est pas un programme ou un code JavaScript)
- Le Responsive web design consiste à utiliser les langages CSS et HTML pour redimensionner, masquer, réduire, agrandir ou déplacer le contenu d'une page pour s'adapter à l'écran d'affichage



CSS: Adapter une page web au dispositif d'affichage

Les media queries:

La règle **@media**, introduite dans CSS2, permet de définir différentes règles de style pour différents types de médias.

Les medias queries dans CSS3 ont étendu l'idée des types de médias CSS2 : examiner la capacité du périphérique au lieu de chercher son type.

Les requêtes multimédias peuvent être utilisées pour vérifier de nombreuses propriétés, telles que :

- Largeur et hauteur de la fenêtre
- Largeur et hauteur de l'appareil
- Orientation (mode paysage ou portrait)
- Résolution

CSS: Adapter une page web au dispositif d'affichage

L'exemple suivant change la couleur d'arrière-plan selon la règle suivante :

- Si la largeur de la fenêtre est 480 px au minimum, la couleur sera bleu clair
- Si la largeur de la fenêtre est 200 px au minimum, la couleur sera lightgreen
- Si la largeur de la fenêtre est inférieure à 200 px, la couleur reste blanche

```
@media screen and (min-width : 200px)
{
    body
    {
        background-color :rgb(144, 174, 238);
    }
}
@media screen and (min-width : 480px)
{
    body
    {
        background-color : lightgreen;
    }
}
```

CSS: Introduction aux frameworks CSS

Un Framework CSS est une bibliothèque préparée et prête à l'emploi. Il facilite le travail du développeur d'interface utilisateur. Plutôt que de commencer chaque projet à partir de zéro, un Framework CSS donne des outils pour créer rapidement une interface utilisateur.

Avantages d'utilisation d'un Framework CSS:

- **Rendre les sites web plus faciles à maintenir** : Les Frameworks CSS utilisent les mêmes identifiants et noms de classes dans vos projets grâce à un code CSS et HTML par défaut. Ceci vous permet de maintenir les sites sans avoir à vous enfoncer dans le code source pour savoir comment il fonctionne.
- **Maximise la compatibilité entre les différents navigateurs** : Les Frameworks CSS fournissent une structure compatible avec tous les navigateurs.
- **Augmente votre productivité** : Si vous développez de nombreux sites similaires, un code CSS générique peut améliorer votre productivité.
- **Fournit un code propre et bien structuré.**

Les trois Framework CSS les plus utilisés:

- **Tailwind CSS**
- **Bootstrap**
- **Bulma**